Convolution Exam Guide – Part 2

Stock Market Smoothing, Moving Averages & Polynomial Multiplication

Plus: Complete Code Analysis & Master Summary

Quick Reference & Concept Guide

Contents

Part I
New Question Types

1 Question Type 4: Exponential Smoothing

1.1 Problem Statement

Given stock prices, a window size n, and a parameter α, perform **exponential smoothing**. Recent prices get exponentially more weight.

1.2 The Core Concept

What is Exponential Smoothing?

Imagine you have 10 days of stock prices. You pick a window of 3 days and give:

- Most recent day: weight $\alpha = 0.8$
- One day before: weight $\alpha(1-\alpha) = 0.16$
- Two days before: weight $\alpha(1-\alpha)^2 = 0.032$

Then multiply each price by its weight and add them up. Slide the window forward and repeat.
This is just convolution with a specific h!

1.3 The Weight Formula

Impulse Response for Exponential Smoothing

$$h[k] = \alpha \cdot (1-\alpha)^k, \quad k = 0, 1, \ldots, n-1$$

For $\alpha = 0.8$, $n = 3$:

$$h[0] = 0.8 \times (0.2)^0 = 0.8 \quad \text{(newest price, biggest weight)}$$
$$h[1] = 0.8 \times (0.2)^1 = 0.16$$
$$h[2] = 0.8 \times (0.2)^2 = 0.032 \quad \text{(oldest price, smallest weight)}$$

1.4 Understanding m, n, and Output Trimming

What are m and n?

- m = total number of stock prices (length of input)
- n = window size (how many prices to average at a time)
- Output count = $m - n + 1$

Example: 10 prices, window size 3 $\rightarrow$ $10 - 3 + 1 = 8$ output values.

Why Do We Trim the Output?

Convolution of x (length m) with h (length n) produces $m+n-1$ values. But many are "junk" (incomplete windows). We only want the **valid** part.
Example with $m = 10$, $n = 3$:

- `y[0]`: uses only 1 price $\rightarrow$ junk
- `y[1]`: uses only 2 prices $\rightarrow$ junk
- `y[2]`: uses all 3 prices $\rightarrow$ **first valid**
- `y[3]` to `y[9]`: all valid
- `y[10]`, `y[11]`: incomplete $\rightarrow$ junk

Valid range: index $n-1$ to $m-1$
In code: `range(n-1, m)` gives indices from $n-1$ up to $m-1$.

1.5 Solution Code

Exponential Smoothing – Complete Solution

```python
def exponential_smoothing(prices, n, alpha):
    m = len(prices)

    # input signal: prices in order
    x = make_signal(0, m-1, prices)

    # impulse response: h[k] = alpha * (1-alpha)^k
    h_values = [alpha * (1 - alpha)**k for k in range(n)]
    h = make_signal(0, n-1, h_values)

    # convolve using Offline 1 code
    y = LTISystem(h).output(x)

    # take only valid part (indices n-1 to m-1)
    result = [y.get_value_at_time(i) for i in range(n-1, m)]
    return result
```

1.6 Step-by-Step Verification

For prices $= [10, 11, 12, 9, \ldots]$, $n = 3$, $\alpha = 0.8$:

$$\begin{aligned} h &= [0.8, 0.16, 0.032] \\ y[2] &= x[0] \cdot h[2] + x[1] \cdot h[1] + x[2] \cdot h[0] \\ &= 10 \times 0.032 + 11 \times 0.16 + 12 \times 0.8 \\ &= 0.32 + 1.76 + 9.6 = 11.68 \quad \checkmark \end{aligned}$$

2 Question Type 5: Simple & Weighted Moving Average

2.1 Problem Statement

Given stock prices and window size n, compute:

1. **Simple (Unweighted) Moving Average**: all prices in window get equal weight
2. **Weighted Moving Average**: recent prices get linearly more weight

2.2 The Core Concept

Two Types of Moving Average

For window size $n = 4$:
Unweighted: every price gets weight $1/n$

$$h = \left[\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}\right] = [0.25, 0.25, 0.25, 0.25]$$

Weighted: newest price gets weight n, oldest gets weight 1, then normalize.

$$\begin{aligned} \text{Raw weights} &= [4, 3, 2, 1] \quad \text{(newest} \rightarrow \text{oldest)} \\ \text{Sum} &= 4 + 3 + 2 + 1 = 10 \\ \text{Normalized} &= [0.4, 0.3, 0.2, 0.1] \end{aligned}$$

Both are just convolution with different h values!

2.3 Critical Detail: Weight Order in h

Why `h[0]`

In convolution:

$$y[n] = \sum_k x[k] \cdot h[n-k]$$

When computing a valid output, $h[0]$ multiplies the **most recent** price, $h[1]$ multiplies the one before that, and so on.
So for **weighted** average:

- `h[0]` = biggest weight = n/sum (for newest price)
- `h[n-1]` = smallest weight = 1/sum (for oldest price)

Formula: $h[k] = (n-k)/\text{sum}$ where sum $= n(n+1)/2$

2.4 Solution Code

Simple Moving Average

```
def unweighted_moving_average(prices, n):
    m = len(prices)
    x = make_signal(0, m-1, prices)

    # all weights equal
    h_values = [1/n] * n
    h = make_signal(0, n-1, h_values)

    y = LTISystem(h).output(x)
    return [y.get_value_at_time(i) for i in range(n-1, m)]
```

Weighted Moving Average

```
def weighted_moving_average(prices, n):
    m = len(prices)
    x = make_signal(0, m-1, prices)

    # h[0]=n/sum (newest, biggest), h[n-1]=1/sum (oldest, smallest)
    total = n * (n + 1) / 2
    h_values = [(n - k) / total for k in range(n)]
    h = make_signal(0, n-1, h_values)

    y = LTISystem(h).output(x)
    return [y.get_value_at_time(i) for i in range(n-1, m)]
```

2.5 Verification

Prices $= [1, 2, 3, 4, 5, 6, 7, 8]$, $n = 4$:

Unweighted ($h = [0.25, 0.25, 0.25, 0.25]$):

$$y[3] = (1 + 2 + 3 + 4) \times 0.25 = 2.5 \quad \checkmark$$

Weighted ($h = [0.4, 0.3, 0.2, 0.1]$):

$$y[3] = 4 \times 0.4 + 3 \times 0.3 + 2 \times 0.2 + 1 \times 0.1 = 3.0 \quad \checkmark$$

3 Question Type 6: Polynomial Multiplication

3.1 Problem Statement

Multiply two polynomials using convolution.

Example: $(3x^2 - 2x + 1) \times (2x^3 - 3x + 1) = 6x^5 - 4x^4 - 7x^3 + 9x^2 - 5x + 1$

3.2 The Core Concept

Why Polynomial Multiplication

When you multiply two polynomials, each coefficient of the result is:

$$c_n = \sum_k a_k \cdot b_{n-k}$$

where $k + (n - k) = n$, meaning the exponents of a and b add up to n.

Compare with convolution:

$$y[n] = \sum_k x[k] \cdot h[n - k]$$

Exact same formula!

- $x[k]$ = coefficients of polynomial A (index = exponent)
- $h[k]$ = coefficients of polynomial B (index = exponent)
- $y[n]$ = coefficients of the product (index = exponent)

3.3 The Catch: Input Order

Must Reverse Input and Output!

The input gives coefficients in **decreasing** exponent order:

`3 -2 1` means $3x^2 - 2x + 1$ (order: x^2, x^1, x^0)

But for convolution, index must match exponent:

- `x[0] = 1` (coefficient of x^0)
- `x[1] = -2` (coefficient of x^1)
- `x[2] = 3` (coefficient of x^2)

So: **reverse** input before convolution, **reverse** output after convolution.

3.4 Output Range

Polynomial Multiplication Output

- Degree of A $= d_1$, Degree of B $= d_2$
- Degree of result $= d_1 + d_2$
- Number of coefficients $= d_1 + d_2 + 1$

- **No trimming needed!** We use ALL convolution outputs.

3.5 Solution Code

Polynomial Multiplication

```python
def polynomial_multiply(coeffs_a, coeffs_b):
    d1 = len(coeffs_a) - 1
    d2 = len(coeffs_b) - 1

    # reverse: so that index = exponent
    a_values = coeffs_a[::-1]
    b_values = coeffs_b[::-1]

    # build signals
    x = make_signal(0, d1, a_values)
    h = make_signal(0, d2, b_values)

    # convolve
    y = LTISystem(h).output(x)

    # extract all coefficients, then reverse back
    result = [y.get_value_at_time(i) for i in range(d1 + d2 + 1)]
    return result[::-1]
```

3.6 Step-by-Step Verification

$A = 3x^2 - 2x + 1$, coeffs $= [3, -2, 1]$
$B = 2x^3 - 3x + 1$, coeffs $= [2, 0, -3, 1]$

After reversing:

$$a = [1, -2, 3] \quad (x[0] = 1,\ x[1] = -2,\ x[2] = 3)$$
$$b = [1, -3, 0, 2] \quad (h[0] = 1,\ h[1] = -3,\ h[2] = 0,\ h[3] = 2)$$

Convolution:

$$y[0] = 1 \times 1 = 1 \quad (\text{coeff of } x^0)$$
$$y[1] = 1 \times (-3) + (-2) \times 1 = -5 \quad (\text{coeff of } x^1)$$
$$y[2] = 1 \times 0 + (-2)(-3) + 3 \times 1 = 9 \quad (\text{coeff of } x^2)$$
$$y[3] = 1 \times 2 + (-2)(0) + 3(-3) = -7 \quad (\text{coeff of } x^3)$$
$$y[4] = (-2)(2) + 3(0) = -4 \quad (\text{coeff of } x^4)$$
$$y[5] = 3 \times 2 = 6 \quad (\text{coeff of } x^5)$$

Result (increasing): $[1, -5, 9, -7, -4, 6]$
Reversed (decreasing): $[6, -4, -7, 9, -5, 1]$ ✓

4 Possible Related Exam Questions

Variations That May Appear

Variation 1: Different Exponential Decay

$$h[k] = \beta^k \quad \text{instead of } \alpha(1-\alpha)^k$$

Same code, just change the formula for `h_values`.

Variation 2: Centered Moving Average Window centered around current point. Example for $n = 3$:

$$y[i] = \frac{x[i-1] + x[i] + x[i+1]}{3}$$

Use $h = [1/3, 1/3, 1/3]$ starting at index -1:

```
h = make_signal(-1, 1, [1/3, 1/3, 1/3])
```

Variation 3: Custom Weight Function Any formula for weights — just change `h_values`. Everything else stays the same.

Variation 4: Polynomial Division (Advanced) Given product and one polynomial, find the other. This is deconvolution. Very unlikely but possible.

Variation 5: Polynomial Evaluation After Multiplication Multiply two polynomials, then evaluate at a specific x value. Just add:

```
# evaluate polynomial with coefficients [c0, c1, ...] at x_val
value = sum(c * x_val**i for i, c in enumerate(result))
```

Variation 6: Moving Median / Moving Max These are NOT LTI operations. If asked, you cannot use convolution. This is a trick question.

Variation 7: Cumulative Sum as Convolution Step response is cumulative sum of impulse response. So:

$$s[n] = \sum_{k=0}^{n} h[k] = (h * u)[n]$$

where $u[n]$ is unit step. In code:

```
u = make_signal(0, N, [1]*(N+1))     # unit step
s = LTISystem(h).output(u)           # cumulative sum
```

Part II
Complete Code Analysis

5 DiscreteSignal Class: Every Method Explained

Constructor: `DiscreteSignal(start_time, end_time)`

What it does: Creates a signal with all zeros from `start_time` to `end_time`.

When to use: When you need an empty signal to fill in later.

```
x = DiscreteSignal(-2, 3)
# creates x[-2]=0, x[-1]=0, x[0]=0, x[1]=0, x[2]=0, x[3]=0
# length = end - start + 1 = 3 - (-2) + 1 = 6
```

`set_value_at_time(t, value)` and `get_value_at_time(t)`

What they do:
- `set`: puts a value at time t (error if t outside range)
- `get`: reads value at time t (returns 0.0 if t outside range)

When to use:
- `set`: building signals one sample at a time
- `get`: reading output values, especially for trimming

```
x.set_value_at_time(2, 5.0)      # x[2] = 5
val = x.get_value_at_time(2)     # val = 5.0
val = x.get_value_at_time(99)    # val = 0.0 (outside range, safe)
```

`shift(k)` — Delay by k

What it does: Creates a new signal where $y[n] = x[n-k]$.
The time range shifts by k: if original is $[a, b]$, new is $[a+k, b+k]$.

When to use:
- Computing first difference: need $x[n-1]$ = `x.shift(1)`
- Checking time invariance: delayed input should give delayed output

```
# x defined over [0, 2] with values [1, 2, 3]
y = x.shift(1)
# y defined over [1, 3] with values [1, 2, 3]
# y[1]=1, y[2]=2, y[3]=3
# meaning y[n] = x[n-1]
```

Common confusion:
- `shift(1)` = delay by 1 = signal moves **right**
- `shift(-1)` = advance by 1 = signal moves **left**

`multiply(scalar)` — Scale Signal

What it does: Multiplies every sample by `scalar`. Time range stays the same.

When to use:
- Scaling a component in superposition
- Negating a signal for subtraction: `.multiply(-1)`

```
x = make_signal(0, 2, [1, 2, 3])
y = x.multiply(0.5)     # y = [0.5, 1.0, 1.5]
z = x.multiply(-1)      # z = [-1, -2, -3]
```

`add(other)` — Add Two Signals

What it does: Sample-wise addition. Automatically handles different time ranges by extending with zeros.

When to use:
- Adding outputs of parallel systems
- Subtraction: `a.add(b.multiply(-1))`
- Combining superposition components

```
a = make_signal(0, 2, [1, 2, 3])    # a[0..2]
b = make_signal(1, 3, [10, 20, 30]) # b[1..3]
c = a.add(b)
# c[0..3] = [1, 12, 23, 30]
# c[0]=1+0, c[1]=2+10, c[2]=3+20, c[3]=0+30
```

`nonzero_samples()` — List Nonzero Values

What it does: Returns list of `(time, value)` for samples where $|value| > 10^{-12}$.
When to use: Used internally by `output_by_superposition`. Rarely used directly.

```
x = make_signal(0, 3, [1, 0, 0, 5])
x.nonzero_samples()  # [(0, 1.0), (3, 5.0)]
```

`times()` — Get Time Range

What it does: Returns `range(start_time, end_time + 1)`.
When to use: Looping over all time indices.

```
x = DiscreteSignal(-1, 2)
list(x.times())  # [-1, 0, 1, 2]
```

6 LTISystem Class: Every Method Explained

Constructor: `LTISystem(impulse_response)`

What it does: Stores $h[n]$ inside the system as `self.h`.
Key insight: Whatever DiscreteSignal you pass becomes the "impulse response." It does not matter if it is actually an impulse response, step response, or another signal. The system just convolves with it.

```
sys = LTISystem(h)   # stores h
# sys.h is now h
```

`output(input_signal)` — THE Main Function

What it does: Computes convolution $y[n] = \sum_k x[k] \cdot h[n-k]$ for all valid n.
This is the MOST important function. Every problem uses it.

```
y = LTISystem(h).output(x)
```

Used for:

- Normal convolution: `LTISystem(h).output(x)`
- Series combination: `LTISystem(h1).output(h2)`
- Any problem: just change what x and h are

`output_at_time(input_signal, n)` — One Sample

What it does: Computes just $y[n]$ for a single time index n.
How it works internally:

```
def output_at_time(self, input_signal, n):
    t = 0.0
    for k in input_signal.times():
        t += input_signal.get_value_at_time(k) * self.h.
            get_value_at_time(n-k)
    return t
```

This is the convolution sum formula directly in code:

$$y[n] = \sum_k x[k] \cdot h[n-k]$$

`output_range(input_signal)` — Get Output Bounds

What it does: Returns `(y_start, y_end)`.

$$y_{start} = x_{start} + h_{start}, \qquad y_{end} = x_{end} + h_{end}$$

When to use: Rarely used directly. `output()` calls it internally.

`output_by_superposition(input_signal)` — Alternative Method

What it does: Same result as `output()`, different algorithm.
For each nonzero $x[k]$, it:

1. Shifts h by k: $h[n-k]$
2. Scales by $x[k]$: $x[k] \cdot h[n-k]$
3. Adds all these shifted copies together

When to use: For verification. Compare with `output()` to check correctness.

7 Helper Functions Explained

`make_signal(start, end, values)`

What it does: Quick way to create a signal with given values.

```
x = make_signal(2, 4, [7, 8, 9])
# x[2]=7, x[3]=8, x[4]=9
```

When to use: Always. This is how you build x and h.

`max_absolute_difference(sig1, sig2)`

What it does: Finds the biggest $|sig1[n] - sig2[n]|$ over all n.
When to use: Verification. If result ≈ 0, two signals match.

```
diff = max_absolute_difference(y1, y2)
# diff close to 0 means y1 and y2 are the same
```

`first_difference(sig)` — You Add This

What it does: Computes $y[n] = sig[n] - sig[n-1]$
When to use:

- Recover h from step response: $h =$ `first_difference(s)`
- Compute Δx: $\Delta x =$ `first_difference(x)`

```
def first_difference(sig):
    return sig.add(sig.shift(1).multiply(-1))
```

8 When to Use What: Decision Guide

Decision Flowchart

Q: I need to convolve two signals
→ `LTISystem(h).output(x)`
Q: I need to subtract two signals
→ `a.add(b.multiply(-1))`
Q: I need to compute $sig[n] - sig[n-1]$
→ `sig.add(sig.shift(1).multiply(-1))`
Q: I need to combine parallel systems
→ `h1.add(h2)`
Q: I need to combine series systems
→ `LTISystem(h1).output(h2)`
Q: I need to scale and add multiple signals
→ Loop with `.multiply(coeff)` and `.add()`
Q: I need to verify two results match
→ `max_absolute_difference(y1, y2)`
Q: I need only certain output values
→ `y.get_value_at_time(i)` with specific indices

Part III
Master Summary of ALL Question Types

9 Every Problem Has the Same Pattern

The Universal Pattern

Every single problem follows these steps:

1. Define x (input signal) using `make_signal()`
2. Define h (impulse response) using `make_signal()`
3. Convolve: `y = LTISystem(h).output(x)`
4. (Sometimes) Trim or reverse the output

The ONLY thing that changes between problems is:

- What formula you use for h
- Whether you trim the output
- Whether you reverse input/output

10 Complete Comparison Table

Problem	h[k] formula	Output handling	Key thing to add
Basic Convolution	Given directly	Use all	Nothing (Offline 1)
Step Response	$h[n] = s[n] - s[n-1]$	Use all	`first_difference()`

Problem	h[k] formula	Output handling	Key thing to add
System Combination (Parallel)	$h_{eq} = h_1 + h_2$	Use all	Just use `.add()`
System Combination (Series)	$h_{eq} = h_1 * h_2$	Use all	`LTISystem(h1).output(h2)`
Superposition	Same h, multiple inputs	Use all	`output_super()` method
Exponential Smoothing	$h[k] = \alpha(1-\alpha)^k$	**Trim:** `n-1` to `m-1`	Build h + trim output
Unweighted MA	$h[k] = 1/n$	**Trim:** `n-1` to `m-1`	Build h + trim output
Weighted MA	$h[k] = (n-k)/\text{sum}$	**Trim:** `n-1` to `m-1`	Build h + trim output
Polynomial Mult.	Coefficients of poly B (reversed)	**Use all + reverse**	Reverse input & output

11 Quick Intuition for Each Problem

One-Line Intuition

1. **Basic Convolution:** Input through system = multiply and add
2. **Step Response:** Difference of s gives h, difference of x gives Δx. Two ways to get y.
3. **Parallel Systems:** Add impulse responses
4. **Series Systems:** Convolve impulse responses
5. **Superposition:** Process each piece separately, then combine
6. **Exponential Smoothing:** Convolution with decaying weights, keep valid part
7. **Moving Average:** Convolution with equal/linear weights, keep valid part
8. **Polynomial Multiplication:** Convolution IS polynomial multiplication, just reverse

12 How to Approach ANY New Problem

Exam Strategy (3 Questions to Ask Yourself)

1. **What is x?**
   - Stock prices? Polynomial coefficients? Input signal?
   - Create it with `make_signal()`
2. **What is h?**
   - Equal weights ($1/n$)? Exponential decay ($\alpha(1-\alpha)^k$)?
   - Linear weights ($(n-k)/\text{sum}$)? Polynomial coefficients?

- Given directly? Recovered from step response?
- Create it with `make_signal()`

3. **What part of the output do I need?**
   - All of it? → just return `y`
   - Valid part only? → trim with `range(n-1, m)`
   - Reversed? → `result[::-1]`

After answering these 3 questions, write:

```
x = make_signal(start, end, x_values)
h = make_signal(start, end, h_values)
y = LTISystem(h).output(x)
# extract what you need from y
```

That is it. Every problem.

13 Common Mistakes to Avoid

Exam Pitfalls

1. **Forgetting to reverse** in polynomial multiplication
   - Input: decreasing exponent order
   - Convolution needs: increasing exponent order (index = exponent)
2. **Wrong weight order** in weighted moving average
   - `h[0]` = weight for **newest** price (biggest)
   - `h[n-1]` = weight for **oldest** price (smallest)
3. **Forgetting to trim** in moving average / smoothing
   - Convolution gives $m + n - 1$ outputs
   - You only want $m - n + 1$ valid outputs
   - Take `range(n-1, m)`
4. **Forgetting to normalize weights**
   - Weighted MA: raw weights $[4, 3, 2, 1]$, must divide by sum $= 10$
5. **Using numpy.convolve**
   - Strictly prohibited. Always use `LTISystem(h).output(x)`
6. **Confusing `set_value_at_time` and `get_value_at_time`**
   - `set` raises error if t outside range
   - `get` returns 0.0 if t outside range (safe)

14 Final Quick Reference Card

Copy This to Your Mind Before Exam

Subtraction:

```
a.add(b.multiply(-1))
```

First Difference:

```
sig.add(sig.shift(1).multiply(-1))
```

Convolve:

```
LTISystem(h).output(x)
```

Parallel:

```
h1.add(h2)
```

Series:

```
LTISystem(h1).output(h2)
```

Verify:

```
max_absolute_difference(y1, y2)
```

Trim (Moving Average / Smoothing):

```
[y.get_value_at_time(i) for i in range(n-1, m)]
```

Reverse (Polynomial):

```
coeffs[::-1]
```

Output Range:

$$y_{start} = x_{start} + h_{start}, \quad y_{end} = x_{end} + h_{end}$$